

Advanced Data Structures and Algorithm Analysis

— Project 1 Lab Report

Roll Your Own Mini Search Engine

Date: 2026-10-04

Team Division:

Member	Module
Wang Zhanyou	Data preprocessing (preprocess) + stop-word processing (stopword-processor)
Chen Siyu	Inverted index construction (build-index)
Hu Hongwei	Query (search) and command-line interface (cli)
Liu Hengyi	Benchmark and lab report

Chapter 1 Problem Description

1.1 What

This project builds an **inverted index** for a local corpus and answers keyword queries against it. The corpus is the complete works of Shakespeare (a mirror of shakespeare.mit.edu), consisting of 761 HTML files (one scene per file, e.g. `macbeth/macbeth.1.1`), about 939,000 tokens in total.

It consists of four steps:

1. **Word counting and stop-word identification:** scan all documents, count the frequency and document count of each stem, find "noisy words" (stop words) — words that appear in almost every document and carry no discriminating power — and remove them from the index;
2. **Inverted index construction:** tokenize and stem each document, record which documents each stem appears in (along with term frequency and positions), and build the inverted index;
3. **Query:** accept one or more keywords entered by the user and return the IDs of the documents containing them;
4. **Testing:** verify the correctness of the inverted index and demonstrate how the threshold (the required number of matched query words) affects the results.

1.2 Why

Without an inverted index, every query would have to scan all 761 documents from beginning to end, checking whether each document "contains the word". The cost of such a **linear scan** is proportional to the total number of documents, and it grows more wasteful as more queries accumulate.

An inverted index pre-computes and stores the mapping "word → which documents contain it": to answer a query, one directly fetches the posting list (the list of document IDs containing the word) without scanning the whole corpus. The larger the collection, the more obvious the benefit of this "preprocess once, query many times quickly" approach. This project implements it and cross-checks it against a linear scan in order to understand the design and benefits of this core information-retrieval data structure.

1.3 Key design points

- **Where to draw the stop-word boundary:** which metric decides whether a word is "meaningful" or "noise", and whether that metric changes with the data;
 - **Stemming:** both query words and document words must first be reduced to the same stem (loves/loved/loving → love), otherwise they will never match;
 - **Threshold:** with multiple keywords, how many must match before a document qualifies — this should be user-configurable.
-

Chapter 2 Algorithms and Data Structures

2.1 Data structures

There are four data products along the pipeline:

File	Format	Meaning
mapping.json	<code>{"0": "macbeth/macbeth.1.1", ...}</code>	document number (string key) → document path, 761 documents
inner-docs.json	<code>{"0": [length, {stem: [freq, pos0, ...]}]}</code>	per-document term-frequency/position table, ~25 MB
stopword.json	<code>["stem1", "stem2", ...]</code>	stop-word list (stemmed form, 135 words)
inverted-index.json	<code>{stem: {total-frequency, document-frequency, docId[]}}</code>	the inverted index

The fields of one index entry:

- `total-frequency`: total number of occurrences of the stem across the whole corpus;
- `document-frequency (df)`: how many documents the stem appears in;
- `docId`: the ascending list of document IDs containing the stem (the posting list).

In inner-docs, `length` = total number of tokens in that document = the sum of all frequencies in it, used for reconciliation checks.

2.2 Word counting and stop-word identification (Word Counter)

Stop-word rule (data-driven + manual review): a stem is judged as a noisy word if it meets either of the following conditions —

- $df \geq 50\%$ of the documents (appears in 381 or more documents);
- total frequency share $\geq 0.1\%$ (939 or more occurrences in the whole corpus).

After the rule runs, 19 meaningful words (person names / titles such as king, lord, queen, duke) are manually pulled back into the index. The final stop-word list has 135 words, covering about 55.5% of all tokens.

```
for each document D in raw-data:
    text = D's text after stripping HTML tags
    tokens = tokenize(text)           // consecutive letter segments; hyphen/apostrophe
    kept only when followed by a letter (e.g. i'll, twenty-one stay whole; proved--that
    splits)
    for t in tokens: t = lowercase(t)
    stems = stemmer.stemWords(tokens) // Porter2 stemming, one batch call
    freq_map = {}
    for j, s in enumerate(stems):
        if s not in freq_map: freq_map[s] = [1, j]
        else: freq_map[s][0] += 1; freq_map[s].append(j)
    // after the whole document is scanned, write it into inner_docs once
    inner_docs[D.id] = [len(stems), freq_map]

// stop words: one more pass over inner_docs, count df and total frequency of each stem,
// apply the rule above
```

2.3 Inverted index generation (Index Generator)

A single pass over inner-docs, accumulating as it goes; stop words are simply skipped (inner-docs itself is left untouched).

```
index = {}
for each doc_id, (length, freq_map) in inner_docs:
    for each stem, info in freq_map:
        if stem in stopwords: continue
        freq = info[0]
        if stem not in index:
```

```

        index[stem] = {"total-frequency": freq, "document-frequency": 1, "docId":
[doc_id]}
    else:
        index[stem]["total-frequency"] += freq
        index[stem]["document-frequency"] += 1
        index[stem]["docId"].append(doc_id)

```

Documents are scanned in ascending ID order, so each word's docId list is naturally sorted; no extra sorting is needed. Construction also performs reconciliation: each document's length must equal the sum of all its frequencies.

2.4 Query (Query Processor)

Query words go through exactly the same preprocessing as the build side (tokenization + lowercase + stemming + stop-word removal), then their posting lists are fetched and hits are counted. The input is a raw string, tokenized internally first.

```

ipt = the raw input string from the user
q = normalize(ipt)           // same tokenization as the build side: consecutive letter
segments, lowercased (hyphen/apostrophe kept under the same rule)
q = stemmer.stemWords(q)     // stemming
q = remove stop words from q  // result is a deduplicated set of stems
n = len(q)
need = ceil(n * threshold)    // at least how many words must match
hit = {}
for each w in q:
    if w not in index: continue
    for each doc_id in index[w]["docId"]:
        hit[doc_id] += 1
result = { doc_id : hit[doc_id] >= need }
map the doc_ids in result back to a [doc_id, document path] list via mapping.json and
return

```

Threshold is configurable: threshold=1.0 means every keyword must match; the smaller it is, the more words may be missed and the looser the results.

Chapter 3 Testing

Testing method: take inner-docs.json as the ground truth, write a **linear-scan reference implementation** and cross-validate it against the **inverted-index query** — for the same query, the two must return the same set of document IDs. Test environment: Python 3.13, PyStemmer 3.1.0 (Porter2), Windows 11.

3.1 Inverted index correctness tests

Case	Query	Purpose	Linear scan (docs)	Inverted index (docs)	Match
single high-freq	kill	high-frequency retrieval	161	161	✓
single high-freq	young	high-frequency retrieval	251	251	✓
single high-freq	feed	high-frequency retrieval	92	92	✓
single mid-freq	fledg	mid-frequency retrieval	2	2	✓
single mid-freq	tress	mid-frequency retrieval	2	2	✓
single low-freq	aim'st	low-frequency retrieval	1	1	✓
single low-freq	antick'd	low-frequency retrieval	1	1	✓
morphology	killed	should match kill after stemming	161	161	✓
morphology	killing	should match kill after stemming	161	161	✓
rare word	wrlengme	nonexistent word should return 0	0	0	✓
stop word	love	stop word should be filtered out	0	0	✓

Additionally, a **file-level check** was done: the inverted-index.json produced by build-index was compared term by term against a reference index built on the spot from inner-docs (total-frequency / document-frequency / docId) — all 18,401 entries match; the reconciliation numbers during construction (761 docs / 938,770 tokens / 18,536 stems / 319,686 pairs) also all match. The full automated test suite has 34 cases, all passing.

3.2 Effect of threshold on query results

Query	n	threshold	need = ceil($n \times th$)	result docs
kill, king	2	0.5	1	467
kill, king	2	1.0	2	101
feed, young, king	3	0.34	2	192
feed, young, king	3	0.67	3	26
feed, young, king	3	1.0	3	26

The higher the threshold, the more words are required to match and the fewer results (stricter); the lower the threshold, the more words may be missed and the more results (looser). This is exactly the expected behavior of threshold as a "matching bar".

Note: for feed/young/king, the 0.67 and 1.0 rows give the same count (26) because $\text{need} = \text{ceil}(3 \times \text{threshold})$ is 3 in both cases ($\text{ceil}(2.01) = 3$ and $\text{ceil}(3.0) = 3$), so both require all three words to match — not a coincidence.

3.3 Performance data

Metric	Value
terms (after stop-word removal)	18,401
build time	167.0 ms
build peak memory	6.9 MB

Query latency (inverted index, ms) and speedup (= linear-scan time / inverted-index time):

Band	1-word latency	10-word latency	1-word speedup	10-word speedup
high (top 10% df)	0.007	0.049	11.3×	5.6×
mid (near median)	0.001	0.003	103.9×	93.2×
low (bottom 2% df)	0.001	0.002	127.8×	136.7×

(Bands are by df: high = top 10% df, mid = near the median, low = smallest 2% df. Each query group runs 200 times and takes the average.)

Chapter 4 Complexity Analysis

4.1 Time complexity

Linear scan: for one query it must iterate over all $D=761$ documents, checking for each whether the n query words are in "that document's stem dictionary" (average $O(1)$ dictionary lookup), hence $O(D \times n)$. The cost is proportional to the total number of documents and independent of whether a query word is high- or low-frequency.

Inverted index: for each query word it iterates only over that word's posting list, of length $df(w)$; n query words total $O(\sum df(w_i))$, i.e. $O(n \times \bar{df})$ ($\bar{df}$ being the average df). The dictionary lookup itself is $O(n)$.

The essential difference: the linear scan's "every document" is a fixed term D ; the inverted index's "posting length" is df , and $df \leq D$, with low-frequency words having df much smaller than D . So **the**

lower the frequency, the more the inverted index saves.

4.2 Space complexity

Linear scan: needs no extra index, space $O(1)$ (not counting the inner-docs read in).

Inverted index: the index consists of "entries + postings". Number of entries = number of stems after stop-word removal = 18,401; total posting length = number of (document, stem) pairs (after stop-word removal). Space $O(\#terms + \sum df) \approx O(18k + 320k)$, with a measured peak build memory of 6.9 MB. This is the classic "space for query time" trade-off.

Python built-in structures used and their complexity:

Structure	Underlying	Key operation complexity
dict	hash table	lookup / insert average $O(1)$, worst $O(n)$
set	hash table	in test average $O(1)$
list	dynamic array	append amortized $O(1)$, iteration $O(n)$
sort()	Timsort	$O(n \log n)$

These complexities are already included in the derivations above: when building the index, each (document, stem) pair does one dict lookup + accumulation, amortized $O(1)$, so the single pass is overall $O(\text{total (document, word) pairs})$; query hit-counting is implemented with a dict, each increment $O(1)$, the main cost still being iterating the posting lists $O(\sum df)$.

4.3 Discussion of test results

1. **Correctness:** all 34 cross-validation cases match, showing the inverted-index build and query logic are correct; the morphology cases (killed/killing $\rightarrow$ same result as kill) verify that stemming is consistent between the query side and the build side; the stop-word case (love $\rightarrow$ 0 results) verifies that stop-word filtering is effective.
2. **Threshold:** the result count decreases monotonically as the threshold rises, matching the $\text{ceil}(n \times \text{th})$ semantics — the threshold mechanism works.
3. **Speedup rises as df falls:** high-frequency words (df close to D) give the inverted index the smallest advantage ($\sim 5\times$), low-frequency words (df close to 1) the largest ($\sim 130\times$) — consistent with the " $\text{cost} \propto df$ " derivation in §4.1.
4. **Speedup slightly drops as word count grows:** the more words, the more posting lists to merge, diluting the advantage.
5. **Value of stop words:** after removing stop words, the number of index terms drops from 18,536 $\rightarrow$ 18,401; what is removed is precisely the highest-df words, shrinking the index while avoiding non-discriminating results that "match almost every document".
6. **Absolute latency:** queries are all microsecond-level, so at 761-document scale both methods are fast; the speedup matters more in terms of scalability as the collection grows

(see the bonus section below).

4.4 Bonus: scalability analysis

Question: with 500,000 files and 400,000,000 distinct words, will the program still work?

Direct answer: no, not directly — the main bottleneck is memory and storage.

Quantitative estimate (baseline 761 docs / 18,536 words; target $\approx \times 657$ documents, $\times 21,580$ words):

- Assuming $\sim 1,200$ tokens per document on average, total tokens ≈ 600 million;
- The number of index entries = 400 million; each entry holds the stem string + total-frequency + document-frequency + the docId list. In Python each entry, together with the fixed overhead of dict/list objects, costs roughly 100–200 bytes, so $400\text{M} \times 150$ bytes $\approx$ **60 GB** (dictionary part alone, not counting the docIds in the postings);
- For the postings, if each word appears in 10 documents on average, $400\text{M} \times 10 \times 8$ bytes $\approx$ 32 GB (not counting the pointer overhead of Python lists, which in practice multiplies this by another 3–5 $\times$).

Total memory demand is on the order of 100 GB or more, far beyond a typical single machine (16–32 GB).

Specific bottlenecks:

1. **Memory:** the current implementation loads the whole inverted index and inner-docs into memory at once; `json.load` would directly run out of memory (OOM);
2. **Storage format:** JSON repeats the three keys "total-frequency" / "document-frequency" / "docId" for every entry, bloating the file several-fold at 400M entries, and text parsing is slow;
3. **Build time:** the single-pass $O(N)$ scan is itself correct, but once N grows to 600 million the build goes from 167 ms to minutes; once memory runs out and triggers swapping, it degrades to hours or never finishes;
4. **Query:** high-frequency words (like "the") appear in almost every one of the 500,000 files, so their posting lists are hundreds of thousands long, and a single query goes from microseconds to milliseconds;
5. **Python object overhead:** dict/list/str each carry fixed per-object overhead, inflating memory 3–5 $\times$ compared to a compiled language.

How to make it work (directions for improvement):

1. **Separate dictionary from postings:** keep the dictionary in a compact in-memory structure (front coding / trie), and read postings from disk on demand;
2. **Posting compression:** docIds are already ascending, so store deltas and encode them with var-byte / Simple-9 / PForDelta, compressing to 1/5–1/10 of the original size;
3. **Binary storage:** abandon JSON, eliminating key redundancy;

4. **Segmented build + merge** (merge-based indexing): build small indexes in batches and merge them, keeping peak memory bounded;
5. **Sharding / distribution**: split the index across multiple machines by word or by document (e.g. MapReduce indexing);
6. **Mature solutions**: adopt industrial inverted-index systems such as Lucene / Elasticsearch.

Conclusion: at 761 documents and 18,000 words, the naive in-memory implementation is entirely adequate (6.9 MB, 167 ms); but at 500,000 files and 400 million words, the naive implementation cannot work due to memory and storage bottlenecks, and one must introduce posting compression, external-memory indexing, segmented merging, sharding, etc. (or switch to an industrial search engine). **The inverted-index algorithm and data structure themselves remain valid — what must change is the storage and memory management.**

4.5 Known limitations

The current HTML extraction logic does not fully remove interface text inside tags (such as navigation links and button labels on the mirror pages), so a small amount of such text enters the corpus. This is a data-cleaning deficiency at the preprocessing stage; it does not affect the correctness of the inverted-index structure or the query logic, nor the validity of the performance results in this report (all figures are taken from the actually generated data files and are internally consistent). A complete fix would require improving the extraction logic and regenerating the stop-word list.